IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya (Deemed University)

Project – High Level Design

on

Cloud Cost Intelligence Platform for Finance

Student Name: Shahasad Sha Ali

Industry Mentor: **Sumit Kumar Shukla** **Guided by:** **Sindhu**

Table of Contents

1. Introduction. 6
   1.1. Scope of the document. 6
   1.2. Intended Audience 6
   1.3. System overview 6
2. System Design. 6
   2.1. Application Design 6
   2.2. Process Flow 6
   2.3. Information Flow. 7
   2.4. Components Design 8
   2.5. Key Design Considerations 8
   2.6. API Catalogue. 8
3. Data Design. 8
   3.1. Data Model 8
   3.2. Data Access Mechanism 9
   3.3. Data Retention Policies 9
   3.4. Data Migration 9
4. Interfaces 9
5. State and Session Management 10
6. Caching 10
7. Non-Functional Requirements 10
   7.1. Security Aspects 10
   7.2. Performance Aspects 10
8. References 11

HLD - Cloud Cost Intelligence Platform for Finance

1. Introduction

This project is a Python-based data analysis system designed for the cloud finance domain. It enables structured analysis of cloud billing data using Pandas-based aggregation and Matplotlib visualisation, requiring no external APIs or internet connectivity. It accepts a cloud billing CSV export (cloud_billing_1year.csv) containing DATE, SERVICE TYPE, USAGE, and COST columns, and produces cost breakdowns by service and by month, a pie chart showing service cost distribution, and a bar chart showing the monthly spending trend across the full year 2025

1.1. Scope of the Document

This document covers the high-level architecture of the Cloud Cost Intelligence Pipeline, including: component interactions and data flow through the four-layer notebook pipeline; data cleaning and transformation logic overview; cost aggregation and groupby design; visualisation output design (pie chart and bar chart); offline deployment and runtime behaviour within Jupyter Notebook; and nonfunctional requirements at a high level.

1.2. Intended Audience

This document is intended for the following audiences:

• Developers building or extending the billing analysis pipeline

• Architects reviewing the data pipeline and component design decisions

• Industry Mentors from IBM evaluating the technical approach

• University Faculty assessing project completeness and analytical depth

• Finance teams or cloud operations personnel interpreting the cost reports

• Project Managers tracking scope and deliverables

1.3. System Overview

The solution is a lightweight offline data analysis application built with Python. It reads a flat CSV billing export, applies cleaning and transformation logic, computes cost aggregations using Pandas groupby operations, and renders Matplotlib charts inline within a Jupyter Notebook environment. The dataset processed contained 1,095 billing rows (3 services × 365 days) covering January 2025 to December 2025, with a total analysed cloud spend of $120,350

Attribute	Detail
System Type	Offline data analysis pipeline — Jupyter Notebook
Connectivity	Fully offline — no internet or external API required
Language	Python 3.x
Libraries	Pandas, Matplotlib (standard data science stack)
Storage	In-memory Pandas DataFrames; local file I/O for input CSV
Dataset	cloud_billing_1year.csv — 1,095 rows × 4 columns (DATE, SERVICE TYPE, USAGE, COST)
Analysis Period	January 2025 – December 2025 (Full Calendar Year)
Total Spend Analysed	$120,350.00 USD

Figure 1: High Level System Architecture — Four-Layer Jupyter Pipeline (Ingest → Clean → Aggregate → Visualise)

2. System Design

2.1. Application Design

The application follows these core design principles:

- Separation of Concerns — each pipeline stage (ingestion, cleaning, transformation, aggregation, visualisation) has a single well-defined responsibility executed in a distinct Jupyter cell block
- Offline-First Design — no external API calls; all data is loaded from a local CSV file, ensuring full functionality without internet connectivity
- Simplicity Over Complexity — uses only Pandas and Matplotlib; no third-party BI frameworks or database dependencies
- Stateful Session — intermediate DataFrames (data, service_cost, monthly_cost) persist in kernel memory across cells, enabling downstream cells to reference upstream outputs without re-computation
- Flat Schema Assumption — the pipeline assumes the four required columns (DATE, SERVICE TYPE, USAGE, COST) are present; column name stripping (str.strip()) adds resilience against whitespace-padded headers
- Notebook-Native UX — designed for sequential Jupyter cell execution with charts rendered inline via plt.show()

2.2. Process Flow

Figure 2: Overall Process Flow — Sequential Cell Execution In [10] through In [35]

Step	Notebook Cell	Action Performed
1	In [10]	Import pandas as pd and matplotlib.pyplot as plt
2	In [25]	Read cloud_billing_1year.csv using pd.read_csv() — returned 1,095 rows × 4 columns
3	In [26]	Stripped whitespace from all column names using data.columns.str.strip()
4	In [27]	Removed rows with missing values using data.dropna()
5	In [28]	Parsed DATE column to datetime64 using pd.to_datetime(format='%d-%m-%Y')
6	In [29]	Derived Month integer column (1–12) from DATE using .dt.month — DataFrame expanded to 5 columns
7	In [31–32]	Computed service_cost via groupby('SERVICE TYPE')['COST'].sum() → EC2: $77,800 | LAMBDA: $23,340 | S3: $19,210
8	In [33]	Computed monthly_cost via groupby('Month')['COST'].sum() → 12-row Series, Jan $3,797.50 to Dec $16,585.00
9	In [34]	Rendered pie chart 'Cost by Service' with autopct='%1.1f%%' — EC2 64.6%, LAMBDA 19.4%, S3 16.0%
10	In [35]	Rendered bar chart 'Monthly Cost' — Month on x-axis, Cost (USD) on y-axis, consistent upward trend Jan–Dec

2.3. Information Flow

Figure 3: Information Flow Diagram — Unidirectional: CSV File → DataFrame → Aggregation → Chart

Source / Input		Processor		Output
cloud_billing_1year.csv	→	pd.read_csv()	→	Raw DataFrame 1,095 × 4
Raw DataFrame	→	dropna(), str.strip(), to_datetime()	→	Clean DataFrame 1,095 × 5
Clean DataFrame	→	groupby('SERVICE TYPE').sum()	→	service_cost — 3-row Series
Clean DataFrame	→	groupby('Month').sum()	→	monthly_cost — 12-row Series
service_cost	→	plt.pie() + autopct	→	Pie Chart: Cost by Service
monthly_cost	→	plt.bar() + axis labels	→	Bar Chart: Monthly Cost Trend

2.4. Components Design

The system is composed of six core pipeline components:

DataLoader — reads cloud_billing_1year.csv using pd.read_csv() and returns a raw 1,095 × 4 DataFrame with columns DATE, SERVICE TYPE, USAGE, COST.

DataCleaner — applies str.strip() to column names to sanitise whitespace, then dropna() to remove incomplete rows, ensuring aggregations are computed only on valid records.

Transformer — converts DATE to datetime64 using pd.to_datetime() and derives a Month integer column (1–12) using .dt.month, expanding the DataFrame to 5 columns.

CostAggregator — executes two independent groupby().sum() operations to produce service_cost (3 rows: EC2, LAMBDA, S3) and monthly_cost (12 rows: Jan–Dec).

ChartEngine — renders a pie chart (service cost share with percentage labels) and a bar chart (monthly spend trend) using Matplotlib's Pyplot interface.

Control Loop (Notebook) — the Jupyter Notebook cell execution sequence orchestrates the full pipeline: Load → Clean → Transform → Aggregate → Visualise.

2.5. Key Design Considerations

01. Column Name Sanitisation — data.columns.str.strip() is applied immediately after loading to remove leading/trailing whitespace from column names. The column 'SERVICE TYPE' contains a space internally, so it must be referenced exactly as written after stripping outer whitespace; failure to strip causes KeyError on groupby.

02. In-Memory Storage — all DataFrames (data, service_cost, monthly_cost) are held in-memory within the Jupyter kernel session. This eliminates I/O latency but means data is lost on kernel restart; a full re-run is required after any restart.

03. Date Format Specification — pd.to_datetime() is called with format='%d-%m-%Y' to match the DD-MM-YYYY string format in the source CSV. Without this explicit format, Pandas may misparse dates (e.g., treating day as month).

04. Duplicate Groupby Execution — in the submitted notebook, service_cost was computed twice (In [31] and In [32]). Results are identical; this is a redundant cell that should be removed in the final clean submission.

05. Fallback Handling — dropna() acts as the fallback quality gate. Any row with a missing COST, DATE, or SERVICE TYPE is silently dropped before aggregation, preventing NaN propagation into the final cost totals.

2.6 API Catalogue

The following internal Pandas and Matplotlib function interfaces define the analytical API of the pipeline:

Figure 4: Internal Function Interface — Deployment View

Function	Library	Purpose in This Project
read_csv(path)	pandas	Load billing CSV into raw DataFrame
columns.str.strip()	pandas	Sanitise column name whitespace
dropna()	pandas	Remove rows with any missing values
to_datetime(col, format=)	pandas	Parse DATE strings to datetime64 using DD-MM-YYYY format
.dt.month	pandas DatetimeIndex	Extract month integer (1–12) from datetime column
groupby(col)[col].sum()	pandas	Aggregate total cost by service or by month
plot(kind='pie', autopct=)	pandas / matplotlib	Render pie chart with percentage labels per slice
plot(kind='bar')	pandas / matplotlib	Render bar chart of monthly cost with x/y axis labels
plt.title() / xlabel() / ylabel()	matplotlib.pyplot	Add chart title and axis labels
plt.show()	matplotlib.pyplot	Render and display chart inline in Jupyter Notebook

3. Data Design

3.1 Data Model

The billing dataset follows a flat daily-granularity model. Each row represents one cloud service's billing record for a single day. With 3 services billed every day for 365 days, the total expected row count is 1,095, which matches the loaded dataset exactly. After cleaning and transformation, the canonical DataFrame schema is:

Column	Original Type	Processed Type	Description
DATE	object (string)	datetime64[ns]	Billing date parsed from DD-MM-YYYY format
SERVICE TYPE	object	object (stripped)	Cloud service name: EC2, S3, or LAMBDA
USAGE	int64	int64	Usage quantity (hours / requests / GB depending on service)
COST	float64	float64	Billed cost in USD — primary aggregation target
Month	— (derived)	int64	Integer 1–12 extracted from DATE via .dt.month

9.2 Actual Aggregation Results

The following cost totals were produced by the notebook and represent the primary analytical outputs:

Service	Annual Cost (USD)	% of Total	Rank
EC2 (Compute)	$77,800.00	64.6%	1st — Highest
LAMBDA (Functions)	$23,340.00	19.4%	2nd
S3 (Storage)	$19,210.00	16.0%	3rd
TOTAL	$120,350.00	100%	—

Month	Month Name	Total Cost (USD)	Month-over-Month Change	Cumulative (USD)
1	January	$3,797.50	— Baseline	$3,797.50
2	February	$4,480.00	+$682.50 (+18.0%)	$8,277.50
3	March	$6,122.50	+$1,642.50 (+36.7%)	$14,400.00
4	April	$7,050.00	+$927.50 (+15.1%)	$21,450.00
5	May	$8,447.50	+$1,397.50 (+19.8%)	$29,897.50
6	June	$9,300.00	+$852.50 (+10.1%)	$39,197.50
7	July	$10,772.50	+$1,472.50 (+15.8%)	$49,970.00
8	August	$11,935.00	+$1,162.50 (+10.8%)	$61,905.00
9	September	$12,675.00	+$740.00 (+6.2%)	$74,580.00

10	**October**	**$14,260.00**	**+$1,585.00 (+12.5%)**	**$88,840.00**
11	**November**	**$14,925.00**	**+$665.00 (+4.7%)**	**$103,765.00**
12	December	$16,585.00	+$1,660.00 (+11.1%)	$120,350.00

Key Insight: Monthly spend grew 336.7% from January ($3,797.50) to December ($16,585.00), with no month showing a cost reduction — indicating a strong and consistent upward trend in cloud resource usage throughout 2025.

3.2 Data Access Mechanism

All data access is file-based and local. The DataLoader reads the input CSV from the Jupyter working directory using pandas.read_csv(). Processed DataFrames are held entirely in-memory (RAM) for the duration of the active kernel session. Chart outputs are displayed inline using plt.show(). No database, remote storage, or network endpoint is accessed at any stage of the pipeline**.**

3.3 Data Retention Policies

The input CSV (cloud_billing_1year.csv) is read-only and is never modified by the pipeline. Intermediate DataFrames exist only within the active Jupyter kernel and are not automatically persisted to disk. In the submitted notebook, charts are rendered inline via plt.show() without explicit file export. For production deployment, plt.savefig() and DataFrame.to_csv() should be used to save outputs to a versioned output directory (e.g., /outputs/2025-Q4/).

3.4 Data Migration

The pipeline processes exported billing snapshots and does not maintain a persistent database, so formal data migration is not applicable. Adapting the pipeline to a different cloud provider's billing format (e.g., GCP Billing Export or Azure Cost Management CSV) requires: (1) remapping column names to the canonical schema (DATE, SERVICE TYPE, USAGE, COST), and (2) adjusting the date format string in pd.to_datetime(). The aggregation and visualisation layers require no changes.

4. Interfaces

The system interfaces exclusively through local file I/O and the Jupyter Notebook interactive environment:

Interface	Type	Format	Description
Billing Input	**File Read**	**CSV**	**cloud_billing_1year.csv — 1,095 rows, 4 columns**
Pie Chart Output	**Inline Display**	**Matplotlib**	**EC2 64.6% | LAMBDA 19.4% | S3 16.0% — rendered in In [34]**
Bar Chart Output	**Inline Display**	**Matplotlib**	**Monthly spend Jan–Dec 2025 — rendered in In [35]**
User Interaction	**Jupyter Notebook**	**Cell execution**	**Sequential cell execution — In [10] through In [35]**

5. State and Session Management

The platform is stateless by design. All processing state resides in the Jupyter kernel's in-memory namespace. Three named objects persisted across cells during the analysis session: data (the primary cleaned DataFrame, 1,095 × 5), service_cost (the 3-row aggregated cost Series), and monthly_cost (the 12-row monthly cost Series). Cells must be executed in sequence (In [10] → In [35]). Executing a visualisation cell before its upstream aggregation cell would raise a NameError. A full kernel restart followed by 'Run All Cells' is the recommended execution pattern for reproducibility.

6. Caching

No explicit caching layer is implemented in the submitted notebook. The following implicit caching behaviours were present during the session:

- In-memory DataFrame caching: Once loaded in In [25], the data DataFrame remained in kernel memory across all subsequent cells without re-reading the source CSV file.
- Duplicate groupby execution: service_cost was computed in both In [31] and In [32]. Pandas executed the groupby operation twice; results were identical. This is an inefficiency to be resolved in the final clean submission by removing the duplicate cell.
- Figure memory: Matplotlib figure objects for both charts were held in memory until kernel shutdown. For extended sessions producing many charts, plt.close() should be called after plt.show() in each chart cell to release figure memory.

7. Non-Functional Requirements

7.1. Security Aspects

- Data Locality — the billing CSV was processed entirely on the local machine; no data was transmitted to any external endpoint or cloud service during the analysis.
- No Credentials Required — the pipeline required no cloud API keys, IAM roles, or authentication tokens; billing data was used solely as a local static file input.
- Sensitive Financial Data — the dataset contains actual cost figures ($120,350 total annual spend); the CSV and notebook output should be stored with restricted file permissions and excluded from public version control repositories.
- No Web Exposure — all chart outputs were rendered inline within Jupyter; no HTTP server or web dashboard was created, eliminating web-layer attack surfaces.

7.2. Performance Aspects

Metric	Observed / Target	Notes
File load — 1,095 rows	< 1 second	Standard read_csv() on a small dataset; negligible I/O time
dropna() + to_datetime()	< 1 second	Vectorised Pandas operations; no row-level Python loops
groupby().sum() × 2	< 1 second	3-group and 12-group aggregations on 1,095 rows are trivial
Pie chart render (In [34])	2–4 seconds	Matplotlib render time; autopct label computation adds minor overhead
Bar chart render (In [35])	1–3 seconds	12-bar chart; standard Matplotlib render time
Total pipeline runtime	< 10 seconds	Full notebook run In [10] to In [35] on standard hardware

8. References

1. IBM Technical Documentation Standards
2. Pandas Documentation — https://pandas.pydata.org/docs/
3. Matplotlib Documentation — https://matplotlib.org/stable/contents.html
4. Python 3 Documentation — https://docs.python.org/3/
5. AWS Cost Explorer User Guide — https://docs.aws.amazon.com/costmanagement/latest/userguide/ce-what-is.html
6. University Project Guidelines 2026